**Departamento de Computación y Sistemas Inteligentes**
**Computación y Estructuras Discretas III**
**Follow-Up 3: Finite State Transducers**

**Name:** _____

**Objetivos — Unidad 1: Lenguajes Regulares y Autómatas**

---

Definir formalmente los distintos tipos de autómatas finitos (deterministas, no deterministas, no deterministas con transiciones lambda, transductores), dar ejemplos, identificar sus elementos y conocer sus aplicaciones. Entender y aplicar las nociones de lenguajes y expresiones regulares, como de autómatas de estado finito, para el reconocimiento de patrones, procesamiento, validación y extracción de texto usando un lenguaje de programación.

**Context**

---

The word `libro` (book in Spanish and Italian) has four European language translations that use the Latin alphabet: `book` (English), `livre` (French), `buch` (German), and `boek` (Dutch). Although each translation differs significantly in spelling, a Finite State Transducer can map the single input string `libro` to all four outputs simultaneously by branching non-deterministically at the very first input symbol.

**Point 1 — Finite State Transducer: Multilingual Word Translation** **[100%]**

---

Design a **non-deterministic FST** that reads the string `libro` and produces **exactly** the following four translations as its output set: `book`, `livre`, `buch`, `boek`. Your FST must not produce any string beyond these four; every accepting path must lead to one of them and no other.

1. **[50%]** Design the FST **by hand on paper**.
   a. Draw the complete state diagram. Label each transition with $u : v$ notation (read $u$, write $v$); use $\lambda$ when a transition produces no output symbol. Mark the initial state with an incoming arrow ($\rightarrow$) and final states with double circles.

      *Hint:* since the four target words share no common prefix, it is natural to branch from $q_0$ using $\varepsilon$-transitions (reading $\lambda$ on input) into four independent sub-paths, each consuming all five symbols of `libro` while emitting the corresponding translation symbol by symbol.

   b. Identify formally the input alphabet $\Sigma$, the output alphabet $\Gamma$, the set of states $Q$, and the set of accepting states $F$.

      *Note on alphabet sizes:* $|\Sigma|$ contains only the letters of `libro`; $|\Gamma|$ contains all letters that appear in any of the four target words.

2. **[50%]** Implement your FST in `pyformlang`.
   a. Build the transducer using `FST.add_transitions()`, `add_start_state()`, and `add_final_state()`. Complete the skeleton below, adding all missing transitions. Use one independent chain of states per target language; name states descriptively (e.g., `'en0'`, `'en1'`, ... for the English path, `'fr0'`, `'fr1'`, ... for French, etc.).

```python
from pyformlang.fst import FST

fst = FST()

# Each path reads 'l','i','b','r','o' and emits the target word.
# English path:  libro -> book
fst.add_transitions([
    ('q0',  'l', 'en0', ['b']),   # l:b

])
```

```python
fst.add_start_state('q0')
fst.add_final_state('en4')

results = list(map(lambda x: "".join(x), list(fst.translate('libro'))))
print(sorted(results))
```

b. Verify your implementation against the test case below. Recall: `translate()` returns a generator of output-symbol lists; use `list(map(lambda x: "".join(x), list(...)))` to collect all outputs as strings. Your result must contain **exactly** the four expected strings — no more, no less.

| **Input** | **Expected outputs (any order)** | **Got** |
|---|---|---|
| `libro` | `['book', 'boek', 'buch', 'livre']` | |

c. Answer the following questions briefly (2–3 sentences each):

   i. Why is this FST *non-deterministic*? At which state(s) does non-determinism arise, and what kind of non-determinism is it (input ambiguity or $\varepsilon$-transitions)?

   ii. What would happen if you tried to translate the string `libra` instead of `libro`? Would `fst.translate('libra')` return any result? Justify your answer.